

Assessment Title: Timetabling System for Nottingham Trent University

Module Name: SOFT20091 - Software Design and Implementation (SDI)

SAMPLE

Submitted by:

Student Name: [xxxx] [T0xxxx]

Date of Submission: xx April 202x

Contents

Chapter 1: Introduction 4
1.1 Purpose of the Report 4
1.2 Aim and Objectives of the Project 4
1.2.1 Aim: 4
1.2.2 Objectives: 4
1.3 Scope of the Project 4
Chapter 2: System Overview 5
2.1 About the Project 5
2.2 Functionality 5
Chapter 3: System Design 6
3.1 Functional and Non-Functional Requirements 6
3.2 Project Management Documentation 7
3.3 Behavioural Design Diagrams 9
3.4 Structural Design Diagrams 12
3.5 Critical Analysis 13
3.6 Implementation Explanation 13
3.7 Evaluation of Non-Functional Requirements 14
Chapter 4: Implementation 15
4.1 Overview of the Implementation Process 15
4.2 Technologies Used 16
4.4 Screenshots of Console Interfaces 17
Chapter 5: Testing 19
5.1 User Acceptance Testing 19
5.2 Screenshots of testing 22
Chapter 6: Documentation 27
6.1 User Manuals 27
Chapter 7: Conclusion 28
7.1 Summary of Project Outcomes 28
7.2 Challenges Encountered 28
7.3 Reflection on the Development Process 29
7.4 Potential Improvements 29

SAMPLE

References 30
Appendices 31

SAMPLE

Chapter 1: Introduction

1.1 Purpose of the Report

This report presents the design, implementation, and evaluation of a C++ console-based timetabling system for Nottingham Trent University (NTU). The system enables administrators to manage academic modules, sessions, groups, rooms, lecturers, and timetables while students can view the timetable schedule and also can search for their schedule by module, lecturer, or room. To ensure that the solution meets the goals from K1 to K4 and S1 to S4 specified in the coursework brief, the report discusses software design concepts, system architecture, and testing techniques.

1.2 Aim and Objectives of the Project

1.2.1 Aim:

To build a maintainable, STL-based console application that reflects NTU's timetable system and providing useful features for both students and the admins.

1.2.2 Objectives:

- Provide an interface for admin users to manage modules, rooms, groups, students, lecturers, and Sessions.
- Allow Student users to view and search their weekly timetable.
- Support session types like Lecture, Lab, Seminar, and allow their creation and editing by administrators.
- Implement timetable conflict detection based on room and time in the timetable.
- Allow student users to export their timetables to CSV format for easy access and use outside the application.
- Use object-oriented programming and STL containers to ensure a modular and scalable design.

1.3 Scope of the Project

- Module management (adding, editing, deleting modules, rooms, groups, and sessions).
- Room and group management, including renaming and capacity edits.
- Student registration and assignment to groups.
- Lecturer management and module/session assignment.
- Timetable creation, editing, and conflict-checking (week 1 to 53).
- Student-side search by module, room, lecturer and CSV export.
- Console-based friendly user interface menu for both roles Admin and Students.
- Data persistence using text files.

SAMPLE

Chapter 2: System Overview

2.1 About the Project

The Timetabling System for Nottingham Trent University is a console-based application developed in C++ to support both administrators and students. Administrators can manage modules, sessions, students, and timetable conflicts, while students can view and search their academic schedules. The system is built using Object-Oriented Programming and STL containers to keep it organised and scalable. It also saves data using file storage to ensure information doesn't get lost while using the application.

2.2 Functionality

Admin Functionalities:

- Add, edit, and delete modules and assign module leaders.
- Create, edit, and delete student groups.
- Register students and assign them to groups.
- Add, edit, and delete rooms and also including capacity and room name.
- Register and manage lecturers and assign them to sessions.
- Define and manage session types such as Lecture, Lab, and Tutorial etc.
- Create and update weekly timetables (Week 1–53) with all session details.
- Check for conflicts in the timetable for example, repeated sessions or room usage.
- Persistently load and save all data for modules, groups, students, sessions, and more.

Student Functionalities:

- Students can view their weekly timetable based on their group ID.
- Students can search for sessions using filters such as module code, lecturer ID, or room ID.
- Export timetables to CSV format for offline viewing and external use.
- Students can view detailed information about each session, such as the lecturer's name, time, room, and location.

SAMPLE

Chapter 3: System Design

3.1 Functional and Non-Functional Requirements

Functional Requirements:

- Admin can add, edit, and delete modules.
- Admin can add, edit, and delete student groups.
- Admin can define and manage session types (Lecture, Lab).
- Admin can register students and assign them to groups.
- Admin can register lecturers and assign them to modules and sessions.
- Admin can add, edit, and delete rooms.
- Admin can create and update timetables from week 1 to 53.
- Admin can check for timetable conflicts when room and time clashes.
- Student can view their weekly timetable.
- Students can search the timetable by module code, room ID, or lecturer ID.
- Students can export the timetable to CSV format.

Non-Functional Requirements:

Performance

- Adding or editing sessions should complete within 1–2 seconds.
- Searching by module/lecturer/room should return results instantly (≤1 sec).

Usability

- Console-based UI with clear menus for Admin and Student users.
- Prompt-based data entry with input validation.

Reliability

- System saves data to persistent files after changes.
- Admin can reload all data at any time.

Security

- Admin and Student access require login with valid credentials.
- AuthManager verifies roles before granting access.

Maintainability & Scalability

- Object-oriented design using C++ classes and STL for modularity.
- Easily extendable to support features like timetabling by semester or a GUI.

Portability

- The system runs on Windows, macOS, and Linux using standard C++.

SAMPLE

Data Integrity

- Session clashes are prevented through conflict-checking.
- Group and lecturer references are validated during session creation.

3.2 Project Management Documentation

Task List:

- Planning and requirements gathering
- Functional and non-functional specification
- System design and UML diagrams
- Implementation of Admin functionalities
- Implementation of Student functionalities
- Data handling and file I/O
- Testing and debugging
- Report writing and final submission

Timeline:

Phase	Duration	Start	End
Planning & Requirements	1 week	03 Mar 2025	09 Mar 2025
Functional & Non-Functional Specs	2 days	10 Mar 2025	11 Mar 2025
System Design & UML Diagrams	1.1 week	12 Mar 2025	20 Mar 2025
Core Implementation	2.5 weeks	21 Mar 2025	08 Apr 2025
Admin/Student UI features	1 week	09 Apr 2025	14 Apr 2025
Data Handling & Validation	3 days	15 Apr 2025	17 Apr 2025
Testing & Debugging	2 days	18 Apr 2025	20 Apr 2025
Documentation & Submission	2 days	21 Apr 2025	22 Apr 2025

SAMPLE

Gantt Chart:

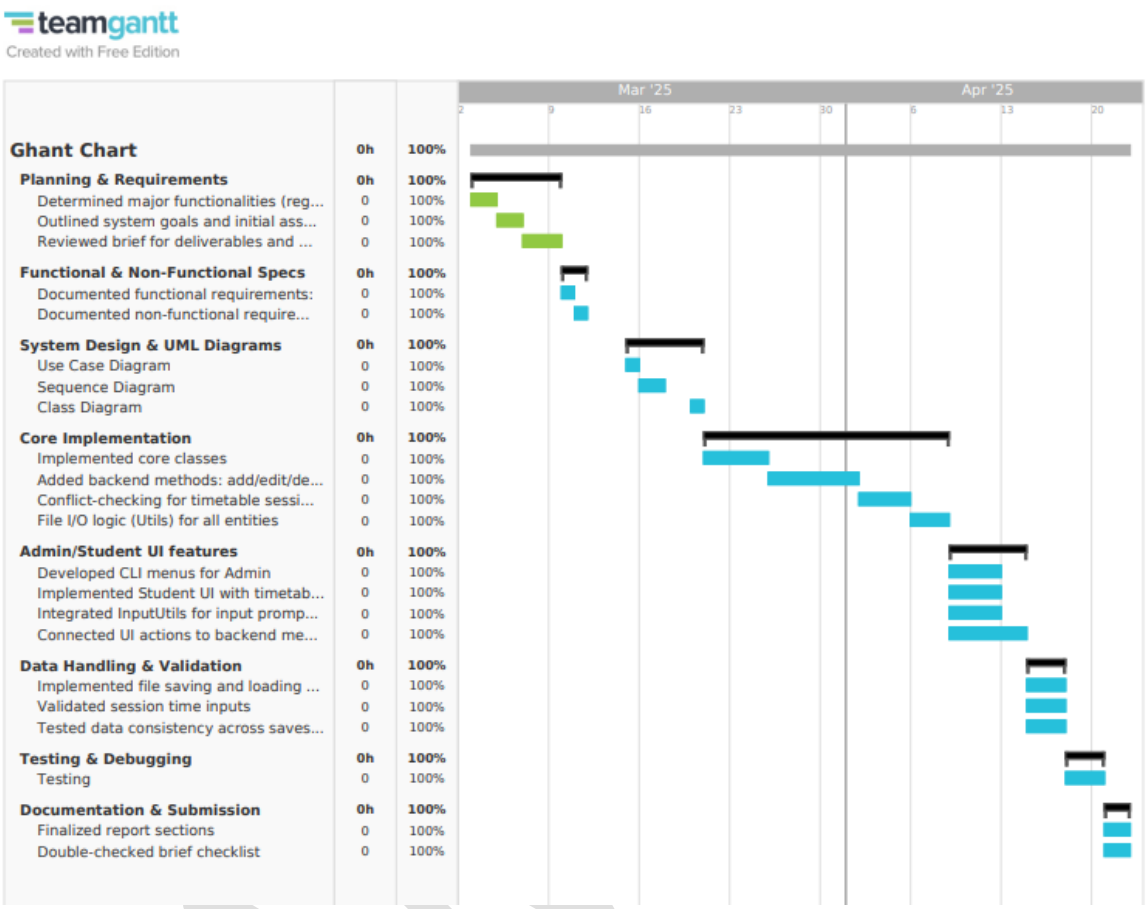


Figure 2 – Gantt Chart

3.3 Behavioural Design Diagrams

Use Case Diagram:

Actors: Admin, Student
Use Cases:

- **Admin**: Manage modules, sessions, students, groups, rooms, timetable and register students, etc, as shown in Figure 1.
- **Student**: View timetable, search timetable, Search Timetable by Lecturer, Search Timetable by Module and export timetable, as shown in Figure 1.

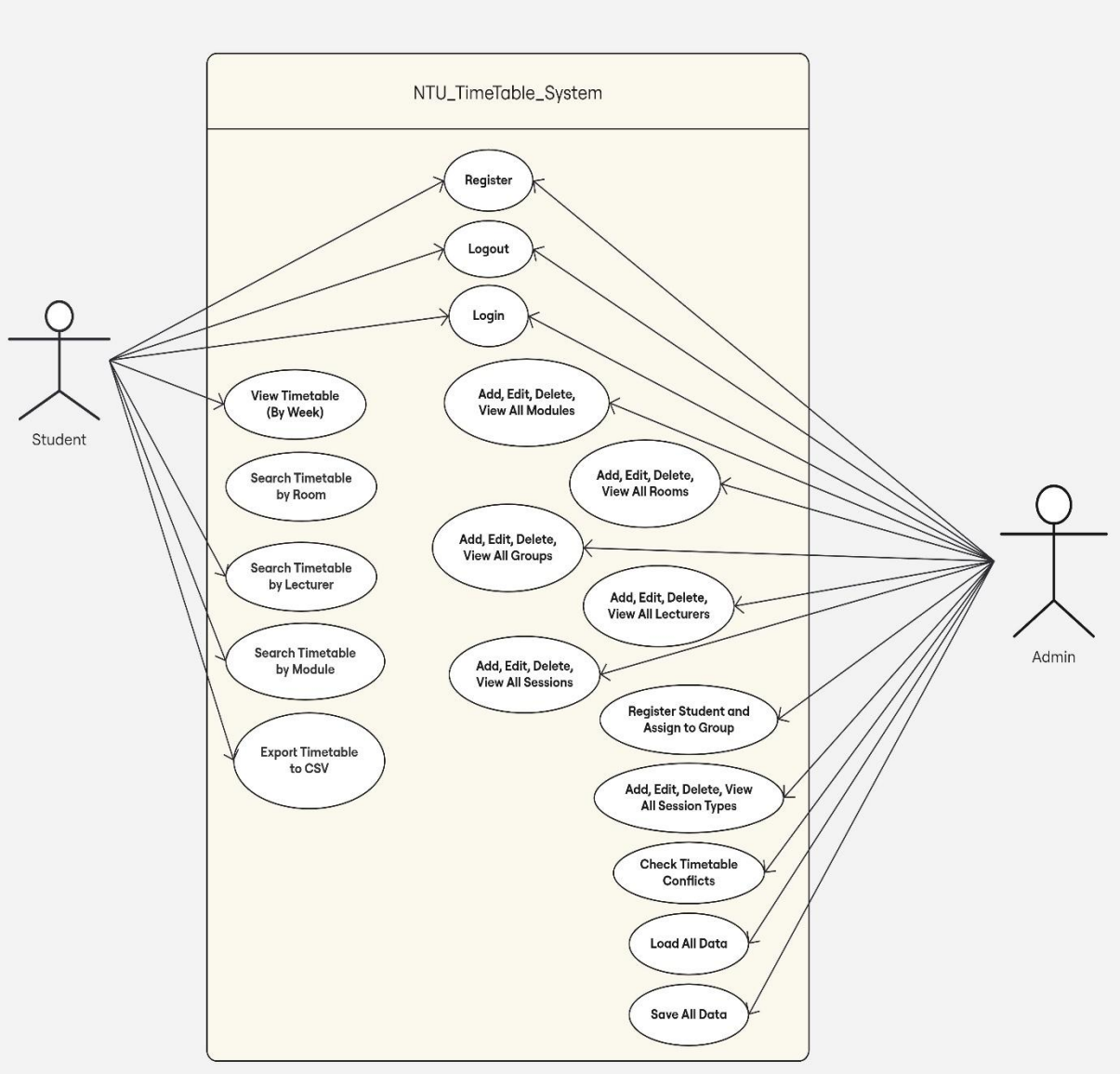


Figure 2 – Use Case Diagram

Sequence Diagram:

AdminUser:

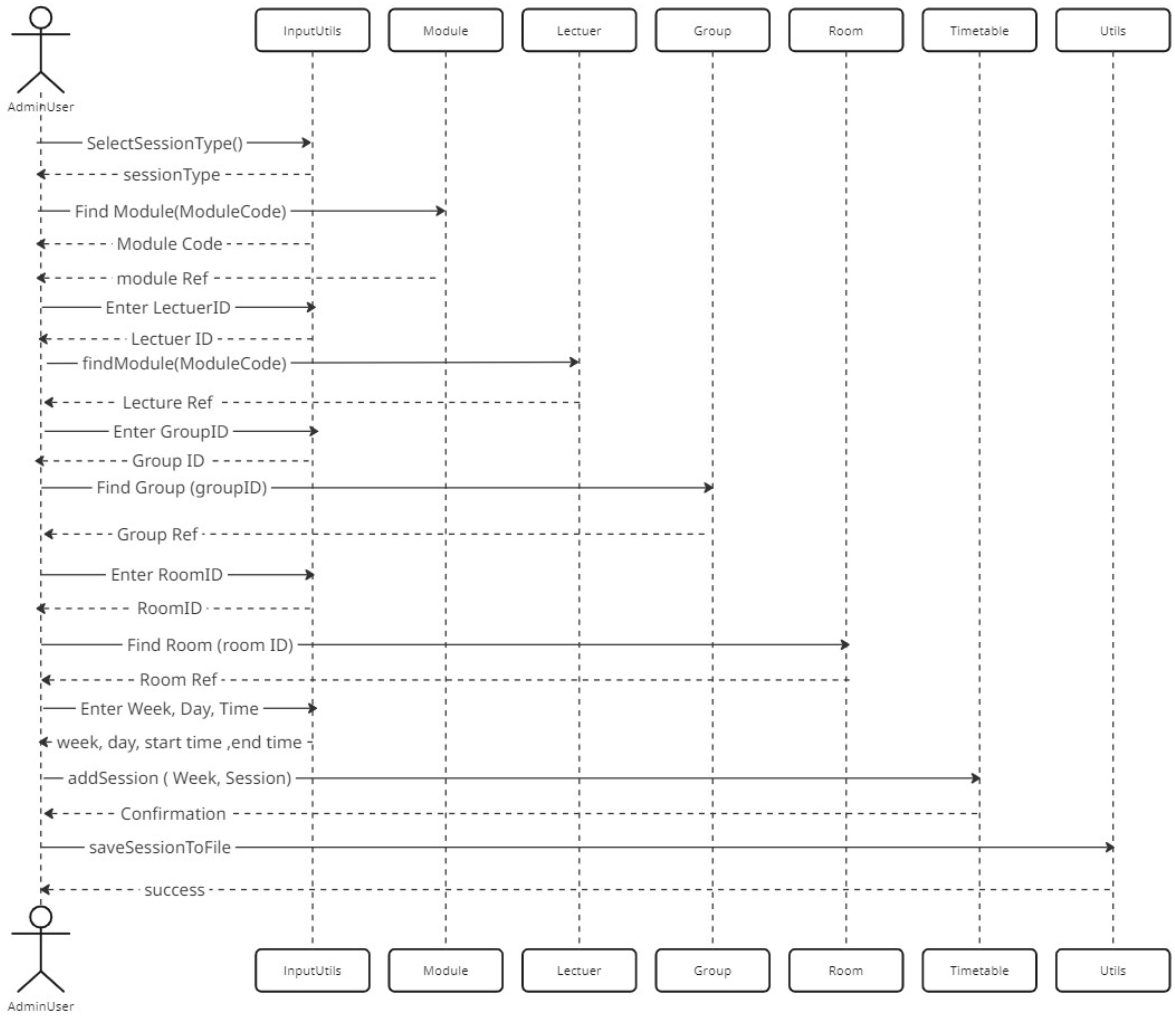


Figure 3 – Sequence Diagram

StudentUser:

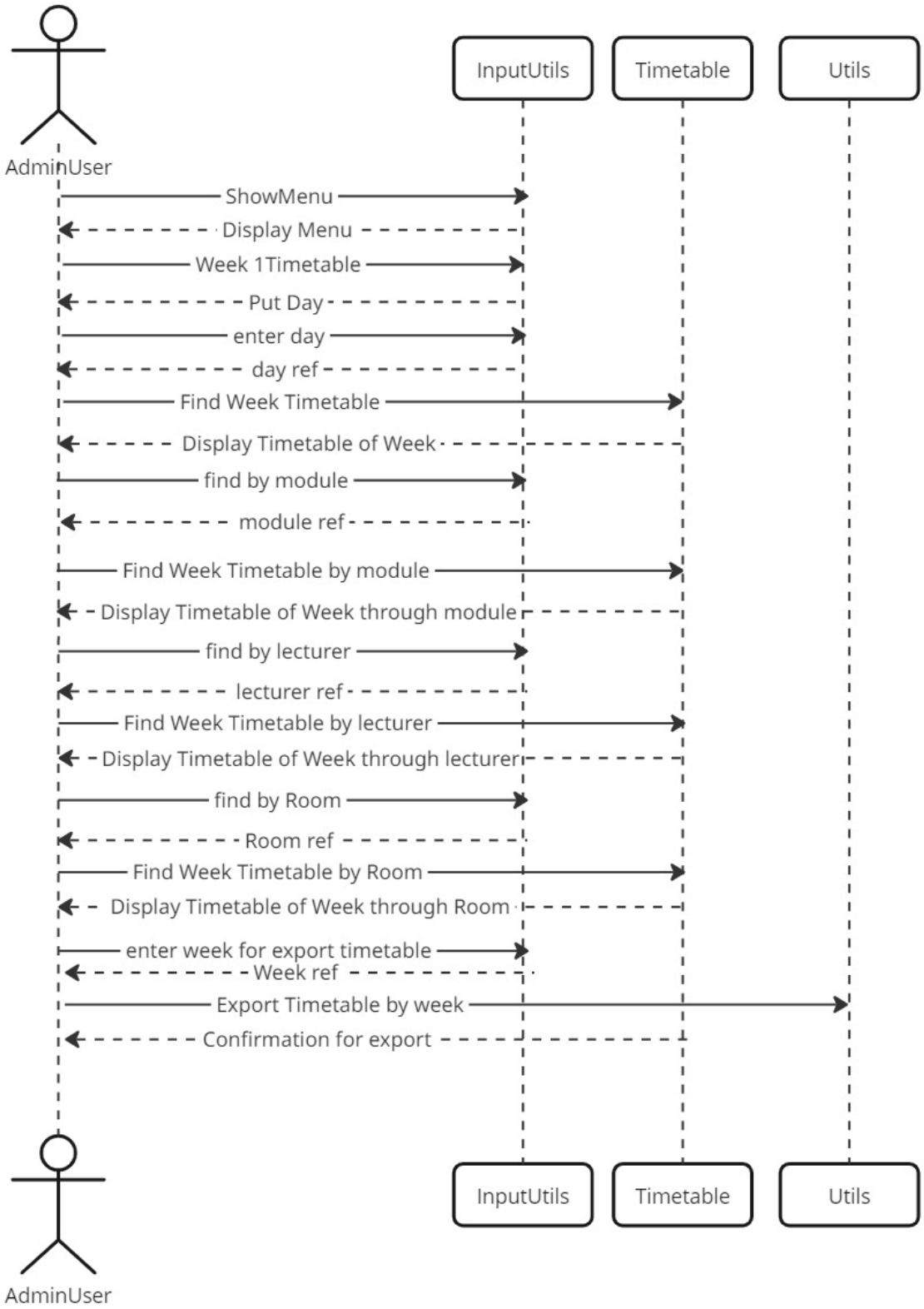


Figure 4 – StudentUser Sequence Diagram

3.4 Structural Design Diagrams

Class Diagram*:*



Figure 5 – Class Diagram

3.5 Critical Analysis

Software Architecture Analysis:

The system uses a simple, well-organised architecture that separates the user interface, core logic, and file handling. Console menus are used for interaction, while the main features are handled by classes like AdminUser and StudentUser. File reading and writing is done in separate helper classes (Utils and AuthManager), making it easy to change or upgrade later. This structure makes the system easy to understand, maintain, and extend. Although it works well for small to medium use, future improvements like switching to a database or adding a better user interface could make it more scalable and user-friendly.

Design Principles Analysis

- **Encapsulation:** Private attributes with getter/setter methods
- **Inheritance:** AdminUser and StudentUser is inherit from the abstract User class.
- **Polymorphism:** User::ShowMenu() is overridden in both subclasses
- **Separation of Concerns:** Each class has a clear, single responsibility for example the Room only focuses on room data and Timetable only manages session by weeks.

3.6 Implementation Explanation

Key Data Structures:

Std::vector:

Stores a dynamic collection of core entities including the modules, rooms, lectures, students, groups, and sessions. It provides easy insertion, deletion, and iteration capabilities suited to frequent CRUD operations in the NTU timetabling system

Std::map:

Maps each week number from week 1 to 53 to a list of corresponding sessions. This structure facilitates efficient retrieval and organisation of weekly timetable and enables rapid access to the session data grouped logically by week.

Classes

AdminUser:

- Manages creation, editing, deletion, and viewing of all module, rooms, lectures,
  groups and session types.
- The Utils class handles all persistent operations, such as saving and loading data from files.
- Contains and manages a Timetable instance, providing functionality for session scheduling, editing, viewing, and comprehensive conflict detection.

SAMPLE

StudentUser:

- Dynamically reloads the student's group information at login from persistent storage to ensure accurate and up-to-date data.
- Provides functionalities for students to view their weekly timetable, as well as perform searches across modules, rooms, or lecturers.
- Supports exporting student-specific timetable data to CSV format through the Utils utility methods.

AuthManager:

- Handles all user authentication and registration processes by reading and writing account data from a centralised text file (Accounts.txt).
- Provides mechanisms to register administrators and students, including ID duplication checks to maintain data integrity.
- Authenticates user login credentials by matching plain-text passwords and user roles.

Timetable:

- Helps organize and manage sessions by week.
- Allows you to add new sessions (addSession), edit existing sessions (getMutableWeek), and view sessions scheduled for a specific week (getSessionsForWeek).
- Checks for scheduling conflicts (checkConflicts) by identifying if two sessions overlap or occur at the same time and place, quickly detecting issues using a simple and fast approach.

Session:

- Stores important scheduling details, including session type, module, lecturer, student group, room, day, start/end times, and week number.
- Provides easy-to-read summaries of session details using the getSummary() method, making it user-friendly.

3.7 Evaluation of Non-Functional Requirements

Reusability: Modular design allows independent reuse of components such as session handling, timetable management, and user authentication. Classes are loosely coupled and can be reused in future academic scheduling systems.

Maintainability: The system follows object-oriented principles with clearly separated responsibilities like AdminUser, StudentUser, and AuthManager. Structured error handling and consistent naming make future maintenance straightforward.

Performance: The use of STL containers such as std::vector and std::map ensures efficient data storage and retrieval. Timetable search and conflict detection run with minimal overhead, even across 53 weeks.

SAMPLE

Usability: A simple, intuitive console-based UI enables easy navigation for both Admin and Student users. Clear prompts, input validation, and feedback messages guide users through every operation.

Portability: The application is written in standard C++ and is compatible with Windows, MacOS, and Linux, making it easy to deploy in different environments.

Chapter 4: Implementation

4.1 Overview of the Implementation Process

Design-to-Implementation Mapping:

- The AdminUser class handles all management functions for modules, students, rooms, groups, lecturers, and sessions.
- The StudentUser class provides features for viewing and searching timetables.
- The Timetable class stores weekly sessions in a std::map<int, std::vector<Session>>.
- The AuthManager class is responsible for user login, registration, and account persistence.

Data Structures Used:

- std::vector: Used to store collections such as modules, students, rooms, groups, and lecturers.
- std::map: Used in the Timetable class to store sessions by week number.
- std::unordered_map in Utils.cpp: Used for fast lookups like for example lecturer ID to name mapping during CSV export.

Sorting and Searching:

- Searching is implemented using std::find_if with lambda functions to match module codes, lecturer IDs, and room IDs.
- Session conflicts are checked by constructing composite keys such as room, day and time, and verifying uniqueness within the same week.

Object-Oriented Programming Concepts Used:

- **Encapsulation**: Each class manages its own data through private members and public methods.
- **Inheritance**: AdminUser and StudentUser both inherit from the abstract base class User.
- **Polymorphism**: User::showMenu() is overridden to implement role-specific menu systems.
- **Composition**: AdminUser holds instances of Timetable, vectors of entities, and a pointer to AuthManager.

SAMPLE

4.2Technologies Used

Programming Language: C++17
Compiler: GCC (g++) and Microsoft Visual C++ (MSVC) via Visual Studio 2022 Community
IDE Used: Visual Studio 2022 Community (configured to use the C++17 standard)
Standard Library Components:

- **Containers:** <vector>, <map>, <unordered_map>
- **Algorithms:** <algorithm> (e.g., std::find_if, std::remove_if)
- **I/O and Persistence:** <fstream>, <sstream>, <filesystem>
- **Utilities:** <tuple>, <string>, <limits>

4.3 Key Features and Functionalities Developed

Admin Console Features:

- **Module Management**: Admins can add, edit, delete modules, and assign lecturers as module leaders.
- **Group & Student Management**: Admins can create student groups, register new students, assign them to groups, and manage group IDs.
- **Room Management**: Admins can add/edit/delete room entries and update capacities and names.
- **Lecturer Registration**: Admins can register and remove lecturers; each lecturer is uniquely identified by an ID.
- **Timetable Management**: Admins can add/edit sessions by selecting session type, module, room, lecturer, group, time, and day.
- **Conflict Checking**: The system detects overlapping sessions based on time, room, and day for a given week.
- **Session Type Editor**: Admins can define new types of sessions or modify/delete existing ones.
- **Data Persistence**: Admins can save or load data from text files covering all core entities and the full timetable.

Student Console Features:

- **Timetable Viewing**: Students can view sessions for any week based on their assigned group ID.
- **Search Functionality**: Students can search for sessions by module code, room ID, or lecturer ID.
- **CSV Export**: Students can export their weekly timetable to a CSV file, including lecturer names and session metadata.

SAMPLE

4.4 Screenshots of Console Interfaces

Admin Main Menu:

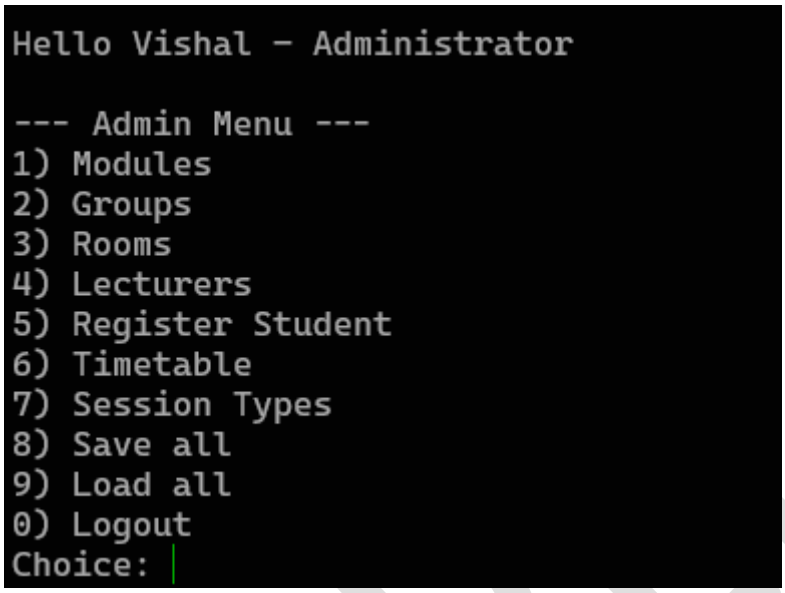


Figure 6 – Admin Menu

Module Menu:

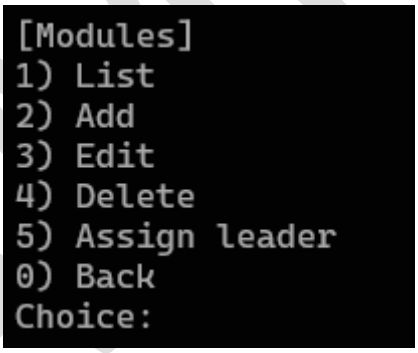


Figure 7 – Module Menu

Timetable conflict detection message:

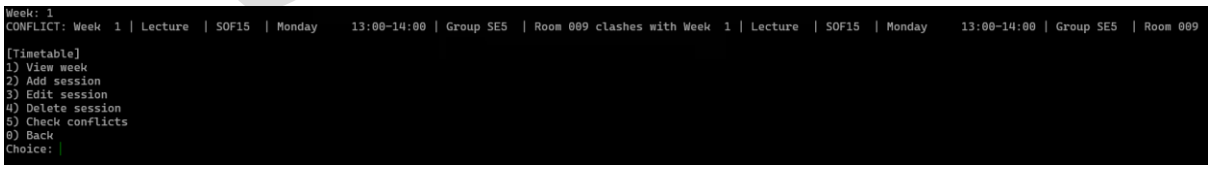


Figure 8 – Conflict Message of the session

SAMPLE

Student Main Menu:

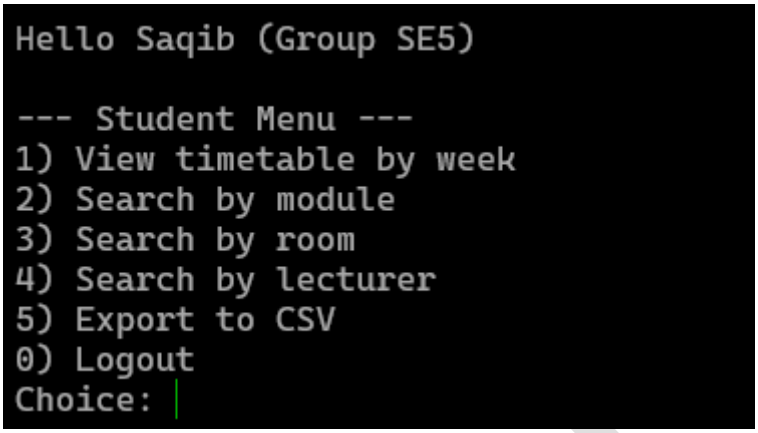


Figure 9 – Student Menu

Student timetable view (Week 1):

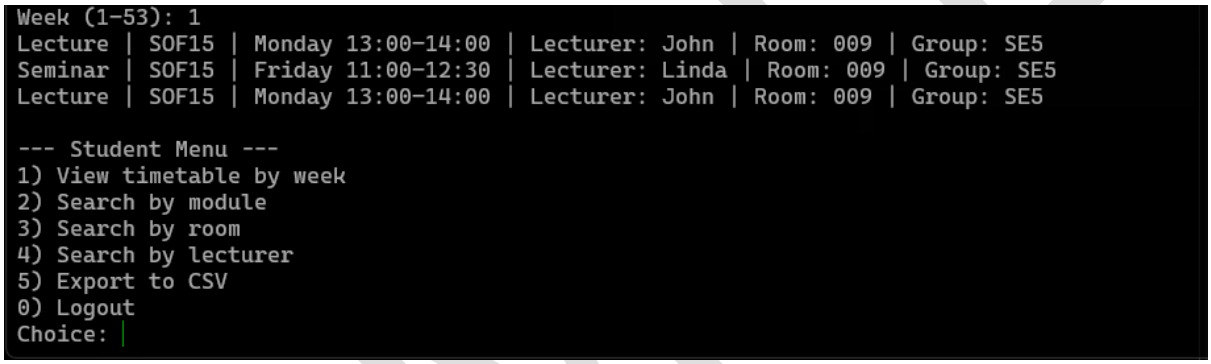


Figure 10 – Viewing Timetable Week 1

CSV export confirmation message:

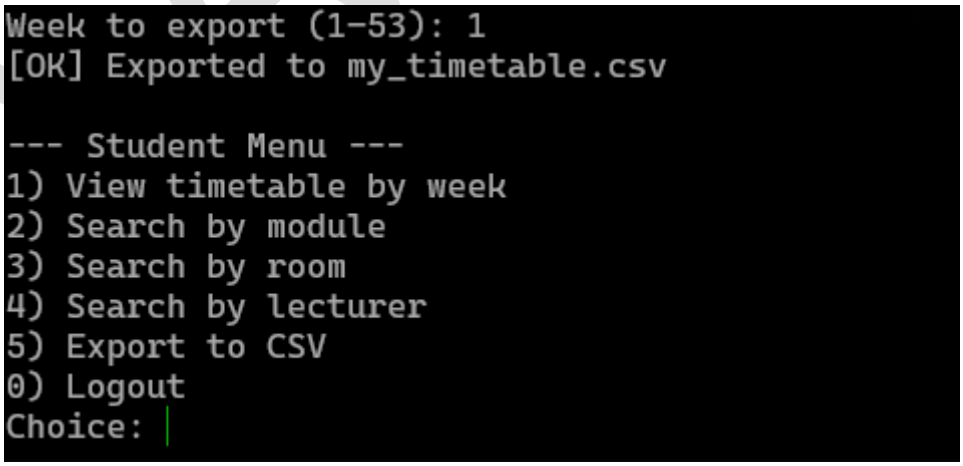


Figure 11 – Export message of Timetable

Chapter 5: Testing

5.1 User Acceptance Testing

Test ID	User Role	Scenario	Expected	Actual
1	Admin	Add a new module	Module added	Module added
2	Admin	List all modules	Display a list of all modules	List displayed
3	Admin	Edit existing module details	Module details updated	Module updated
4	Admin	Delete a module	Module removed from the system	Module deleted
5	Admin	Assign a leader to a module	Lecturer assigned as module leader	Leader assigned
6	Admin	Return to main Admin Menu	Redirect to Admin Menu	Returned to menu
7	Admin	Enter invalid menu choice (e.g., 6)	Show “Invalid choice” error message	Invalid choice shown and ask again.
8	Admin	List all groups	Display list of all groups	Groups listed
	Admin	Add new group	Group created successfully	Group added
10	Admin	Edit existing group details	Group details updated	Group updated
11	Admin	Delete a group	Group removed from system	Group deleted
12	Admin	Add student to a group	Student assigned to selected group	Student added to group
13	Admin	Edit non-existent group	Show “Group not found” error	Error shown
14	Admin	Enter invalid menu choice (e.g., 6)	Show “Invalid choice” error message	Invalid choice shown
15	Admin	Return to main Admin Menu	Redirect to Admin Menu	Returned to menu
16	Admin	List all rooms	Display list of all rooms	Rooms listed
17	Admin	Add new room	Room added successfully	Room added

SAMPLE

18	Admin	Edit the capacity of an existing room	Room capacity updated	Capacity updated
19	Admin	Delete a room	The room was removed from the system	Room deleted
20	Admin	Edit the capacity of a non-existent room	Show “Room not found” error	Error shown
21	Admin	Enter invalid menu choice (e.g., 5)	Show “Invalid choice” error message	Invalid choice shown
22	Admin	Return to main Admin Menu	Redirect to Admin Menu	Returned to menu
23	Admin	List all lecturers	Display list of all lecturers	Lecturers listed
24	Admin	Add new lecturer	Lecturer added successfully	Lecturer added
25	Admin	Delete a lecturer	Lecturer removed from system	Lecturer deleted
26	Admin	Delete non-existent lecturer	Show “Lecturer not found” error	Error shown
27	Admin	Enter invalid menu choice (e.g., 4)	Show “Invalid choice” error message	Invalid choice shown
28	Admin	Return to main Admin Menu	Redirect to Admin Menu	Returned to menu
29	Admin	Register new student with valid details	Student registered successfully	Student registered
30	Admin	Register student with existing Student ID	Show “Student ID already exists” error	Error shown
31	Admin	Return to main Admin Menu after registration	Redirect to Admin Menu	Returned to menu
32	Admin	View timetable for valid week (e.g., week 1)	Display list of sessions for that week	Sessions displayed
33	Admin	View timetable for invalid week (e.g., week 54)	Show error “Invalid week number”	Error shown
34	Admin	Add a new session with valid data	Session added to timetable	Session added
35	Admin	Add a session with invalid module code (e.g., SOF232)	Show “Module not found” error	Module not found
36	Admin	Edit an existing session	Session updated successfully	Session updated
37	Admin	Edit non-existent session	Show “Session not found” error	Error shown
38	Admin	Delete an existing session	Session removed from timetable	Session deleted
39	Admin	Delete non-existent session	Show “Session not found” error	Error shown
40	Admin	Check conflicts in timetable	Display list of conflicting sessions	Conflicts listed

SAMPLE

41	Admin	Return to Admin Menu from Timetable	Redirect to Admin Menu	Returned to menu
42	Admin	List all session types	Display current session types	Session types listed
43	Admin	Add new session type (e.g., Workshop)	New session type added	Session type added
44	Admin	Add duplicate session type	Show “Type already exists” error	Error shown
45	Admin	Edit existing session type (e.g., Lab to Lab1)	Session type updated	Session type edited
46	Admin	Edit non-existent session type	Show “Session type not found” error	Error shown
47	Admin	Delete a session type	Session type deleted	Session type deleted
48	Admin	Enter invalid menu choice (e.g., 5)	Show “Invalid choice” error message	Invalid choice shown
49	Admin	Return to Admin Menu	Redirect to Admin Menu	Returned to menu
50	Admin	Select “Save all”	All data (modules, groups, etc.) saved	Data saved
51	Admin	Select “Load all”	All data loaded into system	Data loaded
52	Admin	Logout from Admin Menu	User logged out and returned to login screen	Logged out
53	Admin	Enter invalid menu choice (e.g., 10)	Show “Invalid choice” error message	Invalid choice shown
54	Student	Login with valid credentials	Welcome message with group info shown	Logged in (Hello Saqib)
55	Student	Login with invalid credentials	Show “Invalid ID or password” error	Error shown
56	Student	View timetable for a valid week (e.g., 1)	Display all sessions for the selected week	Sessions listed
57	Student	View timetable for invalid week (e.g., 54)	Show “Invalid choice: please enter 1–53”	Error message shown
58	Student	Search for sessions by valid module code (SOF11)	Show all sessions across weeks for that module	Sessions listed
59	Student	Search for sessions with invalid module code	Show “Module not found” or empty result	Show “Module not found”
60	Student	Search by valid lecturer ID (e.g., 2)	Show all sessions taught by that lecturer	Sessions listed

SAMPLE

61	Student	Search by invalid lecturer ID	Show “Lecturer not found” or empty result	Message shown: no sessions
62	Student	Logout from Student Menu	Return to main menu	Logged out
63	Student	Search by valid room ID (e.g., 005)	Display all sessions held in that room	Sessions listed
64	Student	Search by invalid room ID (e.g., 43243)	Show “No sessions in room” message	Message shown: no sessions
65	Student	Export timetable for a valid week (e.g., week 1)	Timetable exported to a CSV file	Exported: my_timetable.csv
66	Student	Export timetable with invalid week number (e.g., 5435)	Show “Invalid choice: please enter 1–53” error	Error shown

5.2 Screenshots of testing

Register Section:

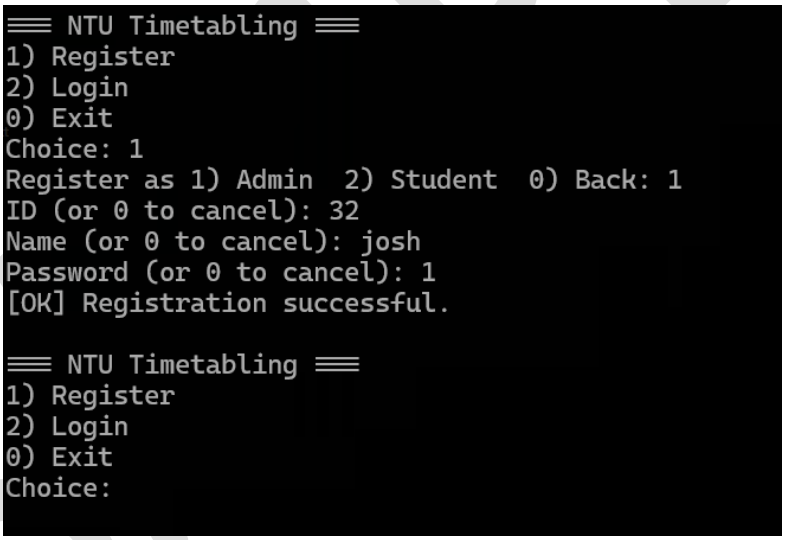


Figure 12 – Registering Admin account

SAMPLE

Admin Menu:

```
--- Admin Menu ---
1) Modules
2) Groups
3) Rooms
4) Lecturers
5) Register Student
6) Timetable
7) Session Types
8) Save all
9) Load all
0) Logout
Choice:
```

Figure 11 – Admin Menu

Module menu:

```
[Modules]
1) List
2) Add
3) Edit
4) Delete
5) Assign leader
0) Back
Choice: 1
Code    Name    Leader
SOF11    Mobile Application Development    Mary
SOF12    DevOps and Continuous Integration    Susan
SOF13    Software Architecture and Design Patterns    James
SOF14    Machine Learning Foundations    John
SOF15    Advanced Topics in Software Engineering    Robert

[Modules]
1) List
2) Add
3) Edit
4) Delete
5) Assign leader
0) Back
Choice:
```

Figure 11 – View Modules

Group menu:

```
[Groups]
1) List
2) Add
3) Edit
4) Delete
5) Add student to group
0) Back
Choice: 1
Group SE1 (0 students)
Group SE2 (0 students)
Group SE3 (0 students)
Group SE4 (0 students)
Group SE5 (1 students)
Group SE6 (0 students)
Group SE7 (0 students)
Group SE8 (0 students)
Group SE9 (0 students)

[Groups]
1) List
2) Add
3) Edit
4) Delete
5) Add student to group
0) Back
Choice:
```

```
[Groups]
1) List
2) Add
3) Edit
4) Delete
5) Add student to group
0) Back
Choice: 2
New group ID: SE10
All data has been saved.
Group added successfully.

[Groups]
1) List
2) Add
3) Edit
4) Delete
5) Add student to group
0) Back
Choice:
```

```
[Groups]
1) List
2) Add
3) Edit
4) Delete
5) Add student to group
0) Back
Choice: 4
Group ID to delete: SE10
Deleted.
All data has been saved.
```

SAMPLE

Lecturer menu:

```
[Lecturers]
1) List
2) Add
3) Delete
0) Back
Choice: 2
Lecturer ID: 11
Name: Josh
All data has been saved.
Lecturer added.

[Lecturers]
1) List
2) Add
3) Delete
0) Back
Choice:
```

```
[Lecturers]
1) List
2) Add
3) Delete
0) Back
Choice: 3
Lecturer ID to delete: 11
Deleted.
All data has been saved.

[Lecturers]
1) List
2) Add
3) Delete
0) Back
Choice:
```

Session Types menu:

```
[Session Types]
1) List
2) Add
3) Edit
4) Delete
0) Back
Choice: 2
New type: Exam
Added.
```

Timetable menu:

```
[Timetable]
1) View week
2) Add session
3) Edit session
4) Delete session
5) Check conflicts
0) Back
Choice: 1
Week (1-53): 1
Sessions for week 1:
[0] Lecture | Module: SOF15 | Monday 13:00-14:00 | Lecturer: John | Group: SE5 | Room: 009
[1] Seminar | Module: SOF15 | Friday 11:00-12:30 | Lecturer: Linda | Group: SE5 | Room: 009
[2] Lecture | Module: SOF15 | Monday 13:00-14:00 | Lecturer: John | Group: SE5 | Room: 009
```

```
  008 (Delta Classroom)
  009 (Epsilon Seminar Room)
  010 (Zeta Workshop)
  011 (Eta Studio)
  012 (Theta Lecture Theatre)
  013 (Iota Lab)
  014 (Kappa Discussion Room)
   ()
Room ID: 005
Week (1-53): 16:00

Select day:
  1) Monday
  2) Tuesday
  3) Wednesday
  4) Thursday
  5) Friday
  6) Saturday
  7) Sunday
Day (1-7): 17:00
Invalid choice: please enter a number between 1 and 7.
Day (1-7): 1
Start time (HH:MM): 17:00
End time (HH:MM): 18:00
All data has been saved.
Session added.
```

```
[Timetable]
1) View week
2) Add session
3) Edit session
4) Delete session
5) Check conflicts
0) Back
Choice: 4
Week (1-53): 1
Sessions for week 1:
[0] Week  1 | Lecture  | SOF15  | Monday     13:00-14:00 | Group SE5  | Room 009
[1] Week  1 | Seminar  | SOF15  | Friday     11:00-12:30 | Group SE5  | Room 009
[2] Week  1 | Lecture  | SOF15  | Monday     13:00-14:00 | Group SE5  | Room 009
Index to delete (0-2): 2
All data has been saved.
Session deleted.
```

Chapter 6: Documentation

6.1 User Manuals

Admin User Manual

The Admin console allows administrators to manage all core entities and build the university timetable. Below are the main functionalities and how to use them:

Modules:

- Add new modules by entering the module code and name.
- Edit module names or assign a lecturer as the module leader.
- Delete unnecessary modules.

Groups & Students:

- Create student groups using unique IDs.
- Register students by entering their ID, name, and password, and assigning them to a group.
- Rename or delete groups and reassign students.

Rooms:

- Add rooms by ID, capacity, and optional name.
- Edit room capacity or name when required.
- Remove rooms that are no longer used.

Lecturers:

- Register new lecturers with a name and unique ID.
- View and delete lecturers as needed.

Timetables:

- Create sessions by selecting session type, module, room, lecturer, group, day, time, and week number.
- Edit session times if changes are needed.
- Save and load the timetable along with all related data.

Session Types:

- Add, rename, or remove types such as Lecture, Lab, Seminar, etc., for flexible scheduling.

Resolving Timetable Conflicts:

- Use the "Check Conflicts" option in the timetable menu to detect overlapping sessions such as two sessions in the same room at the same time.

SAMPLE

- Adjust or edit the conflicting sessions based on feedback.

Student User Manual

The Student console allows enrolled students to interact with their timetable using their group ID. Here's how to use it:

Viewing the Timetable:

- Log in with your student ID and password.
- Select "View Timetable" and choose a week number (1–53).
- All sessions for your group will be displayed in a clear, readable format.

Searching Timetables:

- By Module Code: shows only sessions for a specific module.
- By Room ID: useful for locating where a session takes place.
- By Lecturer ID: shows sessions conducted by a specific lecturer.

Exporting to CSV:

- Choose a week and export your timetable to a CSV file.
- The file will include session type, module, group, lecturer name, room, and times.
- You can open the CSV in Excel.

Chapter 7: Conclusion

7.1 Summary of Project Outcomes

The NTU Timetabling System was successfully developed as a C++ console application. It enables administrators to manage modules, rooms, students, lecturers, and timetables while allowing students to view and search their weekly academic sessions. Key features such as conflict checking, session type customization, and CSV export were fully implemented using object-oriented principles and STL data structures.

7.2 Challenges Encountered

- Managing data consistency across multiple entities like ensuring a student is assigned to a valid group, or a session refers to existing lecturers and rooms.
- Implementing a conflict detection algorithm that checks room/time overlaps efficiently.
- Handling file input/output across different modules while maintaining data integrity.
- Designing a clear, menu-driven user experience that works in the console without GUI elements.

SAMPLE

7.3 Reflection on the Development Process

The use of Object-Oriented Programming made the system highly modular and easy to manage, with classes such as AdminUser, Session, and Timetable effectively encapsulating their respective responsibilities. Additionally, employing the Standard Template Library significantly simplified data handling, particularly through the use of dynamic lists (std::vector) and key-based storage (std::map). Splitting the system into logical components, including authentication, session management, and persistence, further streamlined debugging and testing processes. Moreover, implementing version control using Git facilitated structured development across different phases, enabling efficient tracking of changes throughout the project lifecycle.

7.4 Potential Improvements

Several potential improvements could enhance the system’s functionality and user experience. Implementing a graphical user interface either through a framework like Qt or a web-based frontend would greatly improve usability and make the system more accessible to a wider range of users. Replacing the current file-based storage with a relational database such as SQLite or Mysql would enhance data scalability and support more complex queries. Introducing role-based access control would further strengthen system security by clearly defining permissions for different user roles, such as admin, lecturers, and students. Finally, improving error handling and adding clear logging would help with finding problems and make it easier to maintain the system when it's being used.

SAMPLE

References

W3Schools. (n.d.). W3Schools online web tutorials. [online] Available at: https://www.w3schools.com/ [Accessed 23 March 2025].

GeeksforGeeks. (n.d.). *GeeksforGeeks*. [online] Available at: https://www.geeksforgeeks.org/ [Accessed 23 March 2025].

TeamGantt. (n.d.). *Online Gantt Chart Software – TeamGantt*. [online] Available at: https://app.teamgantt.com/ [Accessed 10 March 2025].

Lucidchart. (n.d.). Lucidchart – Intelligent diagramming for teams. [online] Available at: https://www.lucidchart.com/ [Accessed 20 March 2025].

Miro. (n.d.). Miro – Online collaborative whiteboard platform. [online] Available at: https://miro.com/ [Accessed 16 March 2025].

Microsoft. (n.d.). Visual Studio. [online] Available at: https://visualstudio.microsoft.com/ [Accessed 21 March 2025].

SAMPLE

Appendices

Main.cpp:



```cpp
#include <iostream>
#include <string>
#include "InputUtils.h"
#include "AuthManager.h"
#include "AdminUser.h"
#include "StudentUser.h"

int main() {
    AuthManager auth;
    auth.load();

    // Create a default admin account
    AdminUser adminData("seed", "AdminSeed", "pw", &auth);
    // Load all system data
    adminData.loadAll();

    bool running = true;
    while (running) {
        std::cout << "\n=== NTU Timetabling ===\n"
            << "1) Register\n"
            << "2) Login\n"
            << "0) Exit\n";
        int mainChoice = InputUtils::readInt("Choice: ", 0, 2);

        if (mainChoice == 0) {
            running = false;
            break;
        }

        // register account section
        if (mainChoice == 1) {
            int rolePick;
            do {
                rolePick = InputUtils::readInt(
                    "Register as 1) Admin  2) Student  0) Back: ",
                    0, 2
                );
                if (rolePick == 0) {
                    // back to main menu
                    break;
                }

                std::string id = InputUtils::readLine("ID (or 0 to cancel): ");
                if (id == "0") break;

                std::string nm = InputUtils::readLine("Name (or 0 to cancel): ");
                if (nm == "0") break;

                std::string pw = InputUtils::readLine("Password (or 0 to cancel): "
                if (pw == "0") break;

                bool ok = (rolePick == 1)
```

Module.cpp:



```cpp
#include "Module.h"
#include <iostream>

Module::Module(std::string c, std::string n)
    : code(std::move(c))
    , name(std::move(n))
    , leaderID("")
{
}

std::string Module::getCode() const
{
    return code;
}

std::string Module::getName() const
{
    return name;
}

std::string Module::getLeaderID() const
{
    return leaderID;
}

void Module::setCode(const std::string& c)
{
    code = c;
}

void Module::setName(const std::string& n)
{
    name = n;
}

void Module::setLeaderID(const std::string& id)
{
    leaderID = id;
}

void Module::printSummary() const
{
    std::cout
        << code << " : "
        << name
        << (leaderID.empty() ? " (no leader)" : " (leader " + leaderID + ")")
        << "\n";
}
```

AdminUser.cpp:

```
31      SESSION_TYPES = Utils::loadSessionTypesFromFile("Data/session_types.txt");
32      if (SESSION_TYPES.empty()) {
33          SESSION_TYPES = { "Lecture", "Lab", "Tutorial", "Seminar" };
34          Utils::saveSessionTypesToFile(SESSION_TYPES, "Data/session_types.txt");
35      }
36  }
37
38  // Menu
39  void AdminUser::showMenu() {
40      std::cout << "\nHello " << getName() << " - Administrator\n";
41      int choice;
42      do {
43          std::cout << "\n--- Admin Menu ---\n"
44              "1) Modules\n"
45              "2) Groups\n"
46              "3) Rooms\n"
47              "4) Lecturers\n"
48              "5) Register Student\n"
49              "6) Timetable\n"
50              "7) Session Types\n"
51              "8) Save all\n"
52              "9) Load all\n"
53              "0) Logout\n";
54          choice = InputUtils::readInt("Choice: ", 0, 9);
55          switch (choice) {
56          case 1: moduleMenu();              break;
57          case 2: groupMenu();               break;
58          case 3: roomMenu();                break;
59          case 4: lecturerMenu();            break;
60          case 5: studentRegistrationMenu(); break;
61          case 6: timetableMenu();           break;
62          case 7: sessionTypeMenu();         break;
63          case 8: saveAll();                 break;
64          case 9: loadAll();                 break;
65          }
66      } while (choice != 0);
67  }
68
69  //Student Registration
70  void AdminUser::studentRegistrationMenu() {
71      std::cout << "\n[Register New Student]\n";
72      auto sid = InputUtils::readLine("Student ID: ");
73      auto sname = InputUtils::readLine("Student Name: ");
74      auto pwd = InputUtils::readLine("Password: ");
75      auto gid = InputUtils::readLine("Group ID: ");
76
77      // must have a valid group
78      if (!findGroup(gid)) {
79          std::cout << "Group not found.\n";
80          return;
81      }
82
```

StudentUser.cpp:

```cpp
}

void StudentUser::showMenu() {
    // dynamically reload groupID from students.txt
    auto allStudents = Utils::loadStudentsFromFile("Data/students.txt");
    for (auto& s : allStudents) {
        if (s.getID() == getID()) {
            groupID = s.getGroupID();
            break;
        }
    }

    // pre-load lecturer map
    std::unordered_map<std::string, std::string> lecMap;
    for (auto& L : Utils::loadLecturersFromFile("Data/lecturers.txt"))
        lecMap[L.getID()] = L.getName();

    std::cout << "\nHello " << getName();
    if (!groupID.empty())
        std::cout << " (Group " << groupID << ")";
    std::cout << "\n";

    int choice;
    do {
        std::cout << "\n--- Student Menu ---\n"
            << "1) View timetable by week\n"
            << "2) Search by module\n"
            << "3) Search by room\n"
            << "4) Search by lecturer\n"
            << "5) Export to CSV\n"
            << "0) Logout\n";
        choice = InputUtils::readInt("Choice: ", 0, 5);
        switch (choice) {
        case 1: {
            int wk = InputUtils::readInt("Week (1-53): ", 1, 53);
            auto list = viewTimetableByWeek(wk);
            if (list.empty()) {
                std::cout << "[X] No sessions for week " << wk << ".\n";
            }
            else {
                for (auto& s : list) {
                    std::string lecName = "(unknown)";
                    auto it = lecMap.find(s.getLecturerID());
                    if (it != lecMap.end()) lecName = it->second;

                    // include Group column now
                    std::cout << s.getSessionType()
                        << " | " << s.getModuleCode()
                        << " | " << s.getDay()
                        << " " << s.getStartTime()
                        << "-" << s.getEndTime()
                        << " | Lecturer: " << lecName
                        << " | Room: " << s.getRoomID()
```

TimeTable.cpp:

```cpp
#include "Timetable.h"
#include <stdexcept>
#include <algorithm>

void Timetable::addSession(int wk, const Session& s) {
    if (wk < 1 || wk > 53)
        throw std::out_of_range("Week must be 1 to 53");
    byWeek[wk].push_back(s);
}

std::vector<Session> Timetable::getSessionsForWeek(int wk) const {
    auto it = byWeek.find(wk);
    if (it == byWeek.end()) return {};
    return it->second;
}

std::vector<Session>& Timetable::getMutableWeek(int wk) {
    return byWeek[wk];
}

std::vector<std::string> Timetable::checkConflicts(int wk) const {
    std::vector<std::string> out;
    auto it = byWeek.find(wk);
    if (it == byWeek.end()) return out;

    std::map<std::string, std::string> slotMap;
    for (const auto& s : it->second) {
        std::string key = s.getRoomID() + "_" +
            s.getDay() + "_" +
            s.getStartTime() + "-" +
            s.getEndTime();
        if (slotMap.count(key)) {
            out.push_back("CONFLICT: " + s.getSummary()
                + " clashes with " + slotMap[key]);
        }
        else {
            slotMap[key] = s.getSummary();
        }
    }
    return out;
}

std::vector<Session> Timetable::getAllSessions() const {
    std::vector<Session> all;
    for (auto& [wk, vec] : byWeek) {
        all.insert(all.end(), vec.begin(), vec.end());
    }
    return all;
}

```

InputUtils.cpp:

```cpp
#include <iostream>
#include <limits>
#include <tuple>

namespace InputUtils {

    int readInt(const std::string& prompt, int min, int max) {
        int v;
        for (;;) {
            std::cout << prompt;
            if (std::cin >> v && v >= min && v <= max) {
                std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
                return v;
            }
            std::cin.clear();
            std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
            std::cout << "Invalid choice: please enter a number between "
                << min << " and " << max << ".\n";
        }
    }

    int readInt(int min, int max) {
        // default prompt
        return readInt("Choice: ", min, max);
    }
    //Prompt for a line of text such as names and codes
    std::string readLine(const std::string& prompt) {
        std::cout << prompt;
        std::string s;
        std::getline(std::cin, s);
        if (s.empty()) std::getline(std::cin, s);
        return s;
    }

    std::tuple<std::string, std::string, std::string>
        promptModuleDetails() {
        std::string code = readLine("Module code: ");
        std::string name = readLine("Module name: ");
        std::string leader = readLine("Leader ID: ");
        return std::make_tuple(code, name, leader);
    }

    // Returns true if t matches “HH:MM” with 00≤HH<24 and 00≤MM<60
    bool isValidTime(const std::string& t) {
        if (t.size() != 5 || t[2] != ':') return false;
        int hh, mm;
        try {
            hh = std::stoi(t.substr(0, 2));
            mm = std::stoi(t.substr(3, 2));
        }
        catch (...) {
            return false;
        }
        return hh >= 0 && hh < 24 && mm >= 0 && mm < 60;
    }

}
```